Movie ticket booking

Saturday, 28 February 2026 10:17 AM

↳ BookMyShow

Requirements

- → User can register / login → Auth Service
- → Search movies by
  - → City
  - → Location
  - → Theatre
  - → Language
  - → Date
  - → Name
- → view available shows
- → view seat layout
- → select seats
- → Make payment
- → Receive ticket confirmation.
- → Cancel booking
- → System should lock the seats for 5 minutes to allow for payment. Other user should not be able to book this seat during that time (Reservation)

Admin

- → Add movie
- → Add theatre
- → Add screens
- → Add shows
- → set pricing

Entities

- → User
- → Movie
- → City
- → Theatre
- → Screens
- → Show
- → Seat ✓
- → Booking
- ✓ → ShowSeat / Booking Seat
- → Payment

Relationships

- → City 1 → M Theatres
- Theater 1 → M Screen)
- Screen 1 → M Shows
- Screen 1 → M Seat
- Show 1 → M Booking
- Booking 1 → M ShowSeat / Booking Seat
- Booking 1 — 1 Payment

Constraints :

→ limit no. of seats in one booking to be to prevent system abuse.

Attributes

User
- ↳ id
- ↳ name
- ↳ email
- ↳ password_hash
- ↳ phone
- ↳ created_at
- ↳ updated_at

Movie
- ↳ id
- ↳ title
- ↳ duration
- ↳ genre
- ↳ metadata → Map<> ↳ contains info such as description, cast, director, producer etc.
- ↳ rating

Theatre
- ↳ id
- ↳ name
- ↳ city_id (FK)
- ↳ address
- ↳ lat
- ↳ long

Screen
- ↳ id
- ↳ name
- ↳ theatre_id

Seat
- ↳ id
- ↳ screen_id
- ↳ row
- ↳ seat_number
- ↳ type

Show
- ↳ id
- ↳ movie_id
- ↳ screen_id
- ↳ show_time

ShowSeat
- ↳ id
- ↳ show_id
- ↳ seat_id
- ↳ price
- ↳ status (AVAILABLE / RESERVED / BOOKED)

Seat → physical attributs

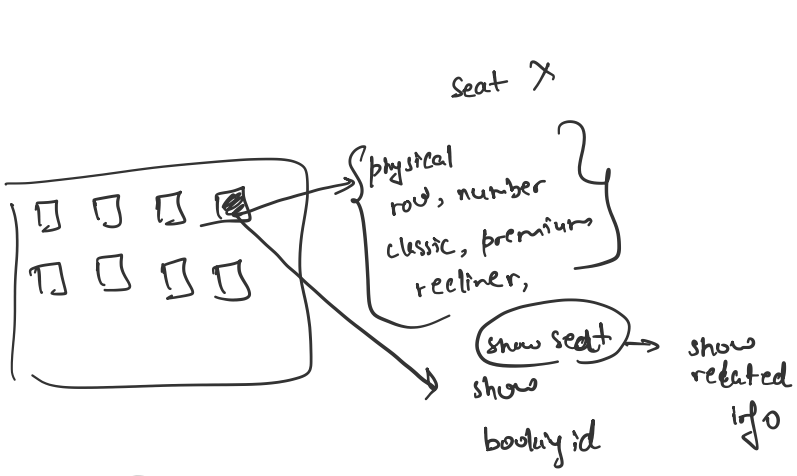
ShowSeat → movie-related attributes & price

Booking
- ↳ id
- ↳ user_id
- ↳ show_id
- ↳ status
- ↳ total_amount

① booking_show_seat_mapping
- ↳ id
- ↳ booking_id
- ↳ show_seat_id

② Booking Seat
- ↳ id
- ↳ booking_id
- ↳ show_seat_id

③ Add booking_id to show_seat



Booking

3NF

show_seat

1 44 53 null → populated at booking time

Booking Seat

Payment
- ↳ id
- ↳ booking_id
- ↳ status
- ↳ transaction_reference
- ↳ created_at
- ↳ updated_at

Services

Movie Search Service

get Movies (List<Filter> filters)
↳ city, language, ⋮

get Movies (city, list<filters>)

Show Service
- → get Shows (movieId, city, date)

Seat Service
- → get Available Seats (showId)
- → lock Seats (userId, showId, seatId[])
- → release Seats (seatId[])

Booking Service
- → create Booking()
- → confirm Booking() → payment
- → cancel Booking

Payment Service
- → initiate
- → confirm

Auth Service
- → signup
- → Login

Movie Service
- ↳ admin operations

Most Important Aspect

Seat locking

- object level locking ↓ Application lock interfaces, example:- Java Lock interface
- DB level locking ↓ maintain state in DB
- Redis level locking ↓ HLD

In conjunction with Transactions ↓ DB level, App level

Transaction is a single unit (which might contain multiple operation) which should be executed as a whole.

transaction {
- read balance
- update balance
- write balance } → error

Transaction Management

↳ ensure that transactions work as intended.

ACID principles should be followed.
- → Atomicity } → as a single unit
- → Consistency } → consistent state
- → Isolation } → each transaction should behave as it was executed alone
- → Durability } → transaction once committed should not be lost

Isolation level
- → Read uncommitted (lowest)
- → Read committed
- → Repeatable Read
- → Serialisable (highest)

A = 1000 —100→ B = 500

{ read A, subtract 100, write A }

{ read B, add 100, write B }

Isolation Levels

① Read uncommitted.

↳ It allows uncommitted data from one transaction to be read in other transactions.

balance = 1000

T1
```
① BEGIN
② UPDATE account
  SET BALANCE = BALANCE - 500
  WHERE ID = XYZ;
     (balance = 500) not committed
```

T2
```
① SELECT balance
  FROM accounts
  where id = XYZ;
  ↳ balance = 500
```
↳ because isolation level is read uncommitted

③ ROLL BACK; ↳ some issue happened

(balance = 1000) ↳ again

T2 = balance is 500

T1 = i never updated

Dirty Read

② Read committed

↳ only committed data from one transaction can be read in other transactions.

Problem #1: Non-Repeatable read.

balance = 1000

T1
```
BEGIN
SELECT balance from accounts
where id = 1
↳ (balance = 1000)
⋮
SELECT balance from accounts
where id = 1
↳ (balance = 700)
```

T2
```
UPDATE accounts
set balance = 700
where id = 1
COMMIT;
```

T1 → first read ⇒ 1000, second read ⇒ 700

non-repeatable read

Problem #2: Phantom Read

Table: loans

id	status
1	PENDING
2	PENDING

T1
```
① BEGIN
② SELECT count(*)
  from loans where
  status = 'PENDING';
  ↳ count = 2
③ SELECT count (*)
  from loans where
  status = 'PENDING';
  ↳ count = 3
```

T2
```
②. INSERT into loans
  values (3, 'PENDING');
  COMMIT;
```

T1 ↳ a new row emerged somehow

Phantom Read

③ Repeatable Read

balance = 1000

T1
```
① BEGIN
② SELECT Balance for
  id = 1
  ↳ balance = 1000
③ Select balance for
  id = 1
  ↳ balance = 1000
    ↳ still
```

T2
```
① Update balance = 700
  for id = 1
  commit.
```

In repeatable read, the transaction will keep on reading the same value for same read statements even if values get changed by other trns meanwhile.

balance = 100, id = 1

balance = 500, id = 2

T1

read for id = 1 ↳ 100

update for id = 2 ↳ balance = 300

read for id = 2 ↳ 500 ↳ 300

Isolation level :- Serializable

Highest form of isolation

Problem first

Let's discuss what happens without serializable

oncall

id	Dev	on call
1	A	TRUE
2	B	TRUE

condition :- at least one dev should be on-call at all time.

A T1
```
① BEGIN
② Select count (*) from on-call
  where on call = TRUE
  ↳ ②
③ UPDATE id = 1 to oncall = FALSE
COMMIT;
```

B T2
```
① BEGIN
② Select count(*)
  from on-call
  where on-call =
        TRUE
  ↳ ②
③ UPDATE id = 2 to
  oncall = FALSE;
  COMMIT;
```

final state

id		
1	A	FALSE
2	B	FALSE

Write Skew
- ↳ can happen in
  - → Read committed
  - → Repeatable read
- ↳ prevented only in
  - → SERIALIZABLE ↳ serializable executes the transactions sequentially (one completes & only then second starts)

hence it prevents writer skew.

Problems
- ↳ Users might see increased no. of failures in high traffic conditions.
- ↳ performance impact due to sequential execution.

Q:- Which transaction level to use for Ticket Booking?

Requirement :- Tickets once booked / reserved should not be available for double booking.

A:-

Option 1

serializable

→ It will never allow double booking

issues:-
- slow performance
- client side errors due to locking
- higher retries.

Option 2

Read committed [with row level locking.

```
select * FROM
show-seat where
id in (x, y, z)
(FOR UPDATE;) -
```
↳ rows are locked.

```
UPDATE show-seat
SET status = 'LOCKED'
where id in (x, y, z);
```

→ COMMIT;

How to do?

Approach #1:- Use stored procedures (older way of doing it)
- → write stored procedures in DB
- → invoke from application

Problems:-
- ↳ no control at application side.
- ↳ more work to do in case of DB migration

Benefits:- ↳ As everything is written in DB, DB can optimize execution perfectly.

Approach #2:- ↳ Fire individual SQL statements from application;

Problems:-
- ↳ cumbersome to write.
- ↳ statement need to be in DB specific form.
- ↳ migration efforts still present.

Approach #3 :- Use transaction management libraries in application.

→ Spring Transaction mgmt (Java)

```
@ Transaction ( isolation = SERIALIZABLE)
{

}
```

Benefits:-
- ↳ no DB code to write
- ↳ easier migration of DB

HLD ↳ write service for seat locking & seat booking.